

```
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
from torch.autograd import Variable  
from torch.optim.lr_scheduler import StepLR  
import numpy as np  
import task_generator as tg  
import os  
import math  
import argparse  
import random
```

These lines import necessary libraries and modules. It imports PyTorch for deep learning, argparse for command-line argument parsing, and other modules for tasks related to data generation, mathematics, and randomization.

```
parser = argparse.ArgumentParser(description="One Shot Visual Recognition")
parser.add_argument("-f", "--feature_dim", type = int, default = 64)
parser.add_argument("-r", "--relation_dim", type = int, default = 8)
parser.add_argument("-w", "--class_num", type = int, default = 5)
parser.add_argument("-s", "--sample_num_per_class", type = int, default = 5)
parser.add_argument("-b", "--batch_num_per_class", type = int, default = 15)
parser.add_argument("-e", "--episode", type = int, default= 1000000)
parser.add_argument("-t", "--test_episode", type = int, default = 1000)
parser.add_argument("-l", "--learning_rate", type = float, default = 0.001)
parser.add_argument("-g", "--gpu", type=int, default=0)
parser.add_argument("-u", "--hidden_unit", type=int, default=10)
args = parser.parse_args()
```

These lines define command-line arguments using argparse to specify hyperparameters for the model. These arguments include feature dimension, relation dimension, class number, sample number per class, batch number per class, training episodes, testing episodes, learning rate, GPU index, and hidden unit size.

Hyper Parameters

FEATURE_DIM = args.feature_dim

RELATION_DIM = args.relation_dim

CLASS_NUM = args.class_num

SAMPLE_NUM_PER_CLASS = args.sample_num_per_class

BATCH_NUM_PER_CLASS = args.batch_num_per_class

EPISODE = args.episode

TEST_EPISODE = args.test_episode

LEARNING_RATE = args.learning_rate

GPU = args.gpu

HIDDEN_UNIT = args.hidden_unit

These lines extract the values of the command-line arguments and store them in variables for use throughout the script.

```

class CNNEncoder(nn.Module):

    # CNN encoder for feature extraction

    def __init__(self):
        super(CNNEncoder, self).__init__()

        # Define the CNN layers

        self.layer1 = nn.Sequential(
            nn.Conv2d(1,64,kernel_size=3,padding=0),
            nn.BatchNorm2d(64, momentum=1, affine=True),
            nn.ReLU(),
            nn.MaxPool2d(2))

        self.layer2 = nn.Sequential(
            nn.Conv2d(64,64,kernel_size=3,padding=0),
            nn.BatchNorm2d(64, momentum=1, affine=True),
            nn.ReLU(),
            nn.MaxPool2d(2))

        self.layer3 = nn.Sequential(
            nn.Conv2d(64,64,kernel_size=3,padding=1),
            nn.BatchNorm2d(64, momentum=1, affine=True),
            nn.ReLU())

        self.layer4 = nn.Sequential(
            nn.Conv2d(64,64,kernel_size=3,padding=1),
            nn.BatchNorm2d(64, momentum=1, affine=True),
            nn.ReLU())

    def forward(self,x):

        # Define the forward pass through the CNN layers

        out = self.layer1(x)

        out = self.layer2(out)

        out = self.layer3(out)

        out = self.layer4(out)

```

return out # Output feature vector of size 64

This code defines a CNN encoder class, which is responsible for feature extraction. The network consists of four convolutional layers with batch normalization and ReLU activations. The forward method defines the forward pass through the CNN layers.

```

class RelationNetwork(nn.Module):

    # Relation network for calculating relations between samples and batches

    def __init__(self, input_size, hidden_size):

        super(RelationNetwork, self).__init__()

        # Define the CNN layers

        self.layer1 = nn.Sequential(

            nn.Conv2d(128,64,kernel_size=3,padding=1),

            nn.BatchNorm2d(64, momentum=1, affine=True),

            nn.ReLU(),

            nn.MaxPool2d(2))

        self.layer2 = nn.Sequential(

            nn.Conv2d(64,64,kernel_size=3,padding=1),

            nn.BatchNorm2d(64, momentum=1, affine=True),

            nn.ReLU(),

            nn.MaxPool2d(2))

        # Define fully connected layers

        self.fc1 = nn.Linear(input_size, hidden_size)

        self.fc2 = nn.Linear(hidden_size, 1)

    def forward(self, x):

        # Define the forward pass through the CNN layers

        out = self.layer1(x)

        out = self.layer2(out)

        out = out.view(out.size(0), -1)

        # Define the forward pass through fully connected layers

        out = F.relu(self.fc1(out))

        out = F.sigmoid(self.fc2(out))

        return out

```

This code defines a Relation Network class, which is used to calculate relations between samples and batches. The network consists of two convolutional layers, each followed by batch

normalization and ReLU activations, and two fully connected layers. The forward method defines the forward pass through these layers.

```

def weights_init(m):

    # Initialize weights of the neural networks

    classname = m.__class__.__name__

    if classname.find('Conv') != -1:

        n = m.kernel_size[0] * m.kernel_size[1] * m.out_channels

        m.weight.data.normal_(0, math.sqrt(2. / n))

        if m.bias is not None:

            m.bias.data.zero_()

    elif classname.find('BatchNorm') != -1:

        m.weight.data.fill_(1)

        m.bias.data.zero_()

    elif classname.find('Linear') != -1:

        n = m.weight.size(1)

        m.weight.data.normal_(0, 0.01)

        m.bias.data = torch.ones(m.bias.data.size())

```

This function **weights_init** is used to initialize the weights and biases of the neural network layers. It loops through the network's layers and initializes the weights according to the type of layer (Convolutional, BatchNorm, or Linear) with specific initialization schemes.


```
def main():
```

```
    # Step 1: init data folders
```

```
    print("init data folders")
```

```
    # init character folders for dataset construction
```

```
    metatrain_character_folders, metatest_character_folders = tg.omniglot_character_folders()
```

The **main** function starts the main part of the script. It initializes the data folders for constructing the dataset by calling the **tg.omniglot_character_folders** function to get folders for characters in the Omniglot dataset.

```
    # Step 2: init neural networks
```

```
    print("init neural networks")
```

```
    feature_encoder = CNNEncoder()
```

```
    relation_network = RelationNetwork(FEATURE_DIM, RELATION_DIM)
```

In this step, the neural networks are initialized. A CNN encoder, **feature_encoder**, is created using the **CNNEncoder** class, and a relation network, **relation_network**, is created using the **RelationNetwork** class. These networks are initialized with specified dimensions.

```
    feature_encoder.apply(weights_init)
```

```
    relation_network.apply(weights_init)
```

The **weights_init** function is applied to the **feature_encoder** and **relation_network** to initialize their weights.

```
    feature_encoder.cuda(GPU)
```

```
    relation_network.cuda(GPU)
```

Both networks are moved to the GPU specified by the **GPU** variable.

```
    feature_encoder_optim = torch.optim.Adam(feature_encoder.parameters(),  
lr=LEARNING_RATE)
```

```
    feature_encoder_scheduler = StepLR(feature_encoder_optim, step_size=100000, gamma=0.5)
```

```
    relation_network_optim = torch.optim.Adam(relation_network.parameters(),  
lr=LEARNING_RATE)
```

```
    relation_network_scheduler = StepLR(relation_network_optim, step_size=100000,  
gamma=0.5)
```

Optimizers are created for the **feature_encoder** and **relation_network**, and learning rate schedulers are set up.

```

if os.path.exists(str("./models/omniglot_feature_encoder_" + str(CLASS_NUM) + "way_" +
str(SAMPLE_NUM_PER_CLASS) + "shot.pkl")):

    feature_encoder.load_state_dict(torch.load(str("./models/omniglot_feature_encoder_" +
str(CLASS_NUM) + "way_" + str(SAMPLE_NUM_PER_CLASS) + "shot.pkl")))

    print("load feature encoder success")

if os.path.exists(str("./models/omniglot_relation_network_" + str(CLASS_NUM) + "way_" +
str(SAMPLE_NUM_PER_CLASS) + "shot.pkl")):

    relation_network.load_state_dict(torch.load(str("./models/omniglot_relation_network_" +
str(CLASS_NUM) + "way_" + str(SAMPLE_NUM_PER_CLASS) + "shot.pkl")))

    print("load relation network success")

```

These lines check if pre-trained model weights exist for the **feature_encoder** and **relation_network** and load them if available.

Step 3: build the training loop

```
print("Training...")
```

```
last_accuracy = 0.0
```

```
for episode in range(EPIISODE):
```

The code enters the training loop. It iterates for a specified number of episodes (defined by **EPIISODE**). The variable **last_accuracy** is used to keep track of the last recorded accuracy for model saving purposes.

```

    feature_encoder_scheduler.step(episode)

    relation_network_scheduler.step(episode)

```

The learning rate schedulers are updated at the beginning of each episode.

Initialize the task and data loaders for samples and batches

```

degrees = random.choice([0, 90, 180, 270])

task = tg.OmniglotTask(metatrain_character_folders, CLASS_NUM,
SAMPLE_NUM_PER_CLASS, BATCH_NUM_PER_CLASS)

sample_dataloader = tg.get_data_loader(task, num_per_class=SAMPLE_NUM_PER_CLASS,
split="train", shuffle=False, rotation=degrees)

batch_dataloader = tg.get_data_loader(task, num_per_class=BATCH_NUM_PER_CLASS,
split="test", shuffle=True, rotation=degrees)

```

In each episode, a task is randomly created by selecting a set of character folders from the training dataset. Two data loaders, **sample_dataloader** and **batch_dataloader**, are created to

load the data for the few-shot learning task. The **rotation** variable is used to randomly rotate the images.

Load sample and batch data

samples, sample_labels = sample_data_loader.__iter__().next()

batches, batch_labels = batch_data_loader.__iter__().next()

The code loads the samples and batches of data from the data loaders. The **samples** are used for comparing with the **batches** in the few-shot learning task.

Calculate features for samples and batches

sample_features = feature_encoder(Variable(samples).cuda(GPU))

sample_features = sample_features.view(CLASS_NUM, SAMPLE_NUM_PER_CLASS, FEATURE_DIM, 5, 5)

sample_features = torch.sum(sample_features, 1).squeeze(1)

batch_features = feature_encoder(Variable(batches).cuda(GPU))

The CNN encoder is applied to both samples and batches to extract their features. These features are reshaped and processed for further comparison.

Calculate relations between samples and batches

sample_features_ext = sample_features.unsqueeze(0).repeat(BATCH_NUM_PER_CLASS * CLASS_NUM, 1, 1, 1, 1)

batch_features_ext = batch_features.unsqueeze(0).repeat(CLASS_NUM, 1, 1, 1, 1)

batch_features_ext = torch.transpose(batch_features_ext, 0, 1)

relation_pairs = torch.cat((sample_features_ext, batch_features_ext), 2).view(-1, FEATURE_DIM * 2, 5, 5)

relations = relation_network(relation_pairs).view(-1, CLASS_NUM)

The code calculates the relations between the samples and batches. It prepares the data for input into the relation network, where it will predict the class relationships. The results are stored in the **relations** tensor.

Calculate loss and perform backpropagation

mse = nn.MSELoss().cuda(GPU)

```
one_hot_labels = Variable(torch.zeros(BATCH_NUM_PER_CLASS * CLASS_NUM,  
CLASS_NUM).scatter_(1, batch_labels.view(-1, 1), 1)).cuda(GPU)
```

```
loss = mse(relations, one_hot_labels)
```

The code calculates the loss using mean squared error (MSE) and prepares the one-hot labels for comparison. The loss is computed based on the differences between predicted relations and actual labels.

```
# Backpropagation and gradient clipping
```

```
feature_encoder.zero_grad()
```

```
relation_network.zero_grad()
```

```
loss.backward()
```

```
torch.nn.utils.clip_grad_norm(feature_encoder.parameters(), 0.5)
```

```
torch.nn.utils.clip_grad_norm(relation_network.parameters(), 0.5)
```

```
feature_encoder_optim.step()
```

```
relation_network_optim.step()
```

Backpropagation is performed, and gradients are clipped to prevent exploding gradients. The model parameters are updated using the optimizer.

```
# Print loss for every 100 episodes
```

```
if (episode + 1) % 100 == 0:
```

```
    print("episode:", episode + 1, "loss", loss.data[0])
```

The code prints the loss every 100 episodes to monitor training progress.

Network Architecture:

- **CNNEncoder:** A convolutional neural network (CNN) encoder is defined for feature extraction. It consists of four convolutional layers with batch normalization and ReLU activation functions. This network extracts features from input images.
- **RelationNetwork:** Another CNN-based network is defined for computing relations between samples. It consists of two convolutional layers followed by fully connected layers.

- A mean squared error loss is calculated between the predicted relations and one-hot encoded labels.
- The loss is printed every 100 episodes.

Testing (Model Evaluation):

- After every 5000 episodes, model evaluation is performed.